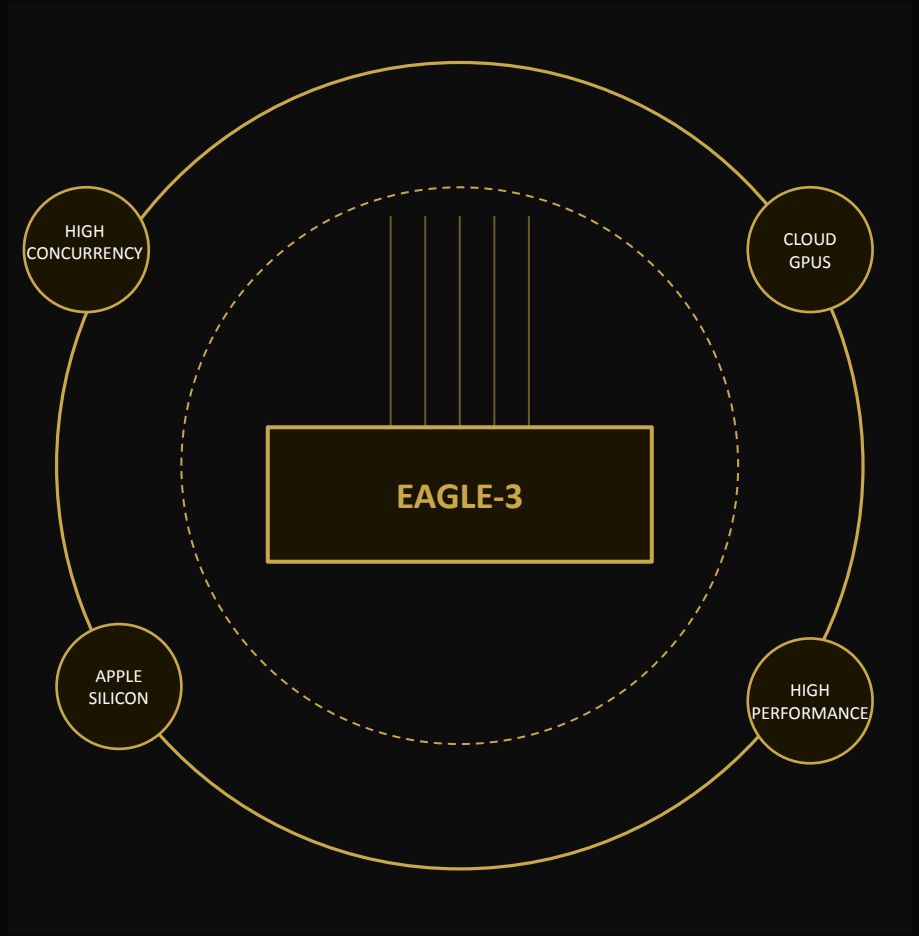


CHARACTERIZING EAGLE-3

SPECULATIVE DECODING IN vLLM

Performance across Cloud GPUs and Apple Silicon under high Concurrency

- Aaryaman
- Shreya Banga
- Rajvardhan
- Himanshu Jhawar (hj2713)

BREAKING THE MEMORY BANDWIDTH BOTTLENECK

THE PROBLEM

LLM inference is memory-bandwidth bound.

Most time is spent moving weights from VRAM to compute cores — not actually computing.

SPECULATIVE DECODING

A small draft model guesses multiple tokens.

The large 8B target model verifies them in parallel.

Accepted tokens = free speedup.

WHY EAGLE-3?

Uses a lightweight feature-level head rather than a separate draft model.

Higher acceptance rates → more tokens per verification pass.

EAGLE-3 PIPELINE:

DRAFT TOKENS → VERIFY (8B) → ACCEPT → SPEEDUP ACHIEVED

RIGOROUS CROSS-PLATFORM MATRIX

3 Tasks × 3 Hardware Tiers × 5 Concurrency Levels = 45 Total Configurations

A100 (80GB)

High-end cloud GPU
HBM2e memory

L4 (24GB)

Cost-efficient cloud GPU
GDDR6 memory

Mac (Unified Memory)

Apple Silicon
Shared memory architecture

TARGET MODEL: Llama-3.1-8B-Instruct (Greedy Decoding)

Chat	ShareGPT
Code	HumanEval
Summarization	CNN

CONCURRENCY LEVELS: 1 | 4 | 8 | 16 | 32 simultaneous users

⚠️ **THE CONSTRAINT: L4 (24GB VRAM)**

Limited 24GB VRAM required strict sequential execution to prevent OOM crashes.

Baseline ~20GB | EAGLE-3 ~22GB | Sequential runner: Run → Kill → Clean VRAM → Next config

UNCOVERING THE vLLM CHUNKING BUG

1

THE SCARE

Initial data showed EAGLE-3 was slower than baseline.

Throughput: Baseline 100 TPS vs EAGLE-3 72 TPS

EAGLE-3 looked worse!

2

THE INVESTIGATION

Raw logs: chars/chunk heavily skewed.

Baseline: 7.7 chars/chunk
EAGLE-3: 3.8 chars/chunk

vLLM batches multiple accepted tokens into ONE SSE chunk — counting chunks undercounted EAGLE-3 by ~2×

3

THE FIX

REQUEST PATCH (LOAD TESTER)

```
stream_options: {
  include_usage: true
}
```

↓

SERVER RESPONSE (AUTHORITATIVE)

```
{"usage": {
  "prompt_tokens": 512,
  "completion_tokens": 256,
  "total_tokens":  768
}}
```

ACCURATE TOKEN COUNTS

Correct metrics. Experiment saved.

AFTER THE FIX:

- ✓ Throughput 2×–3× higher
- ✓ True speedup revealed (not 0.5×)
- ✓ Data integrity restored with server counts
- ✓ Results we can trust and publish

EAGLE-3 IS AN UNCONDITIONAL WIN

Speedup Heatmap: EAGLE-3 TPS ÷ Baseline TPS | All values > 1.0 = EAGLE-3 wins

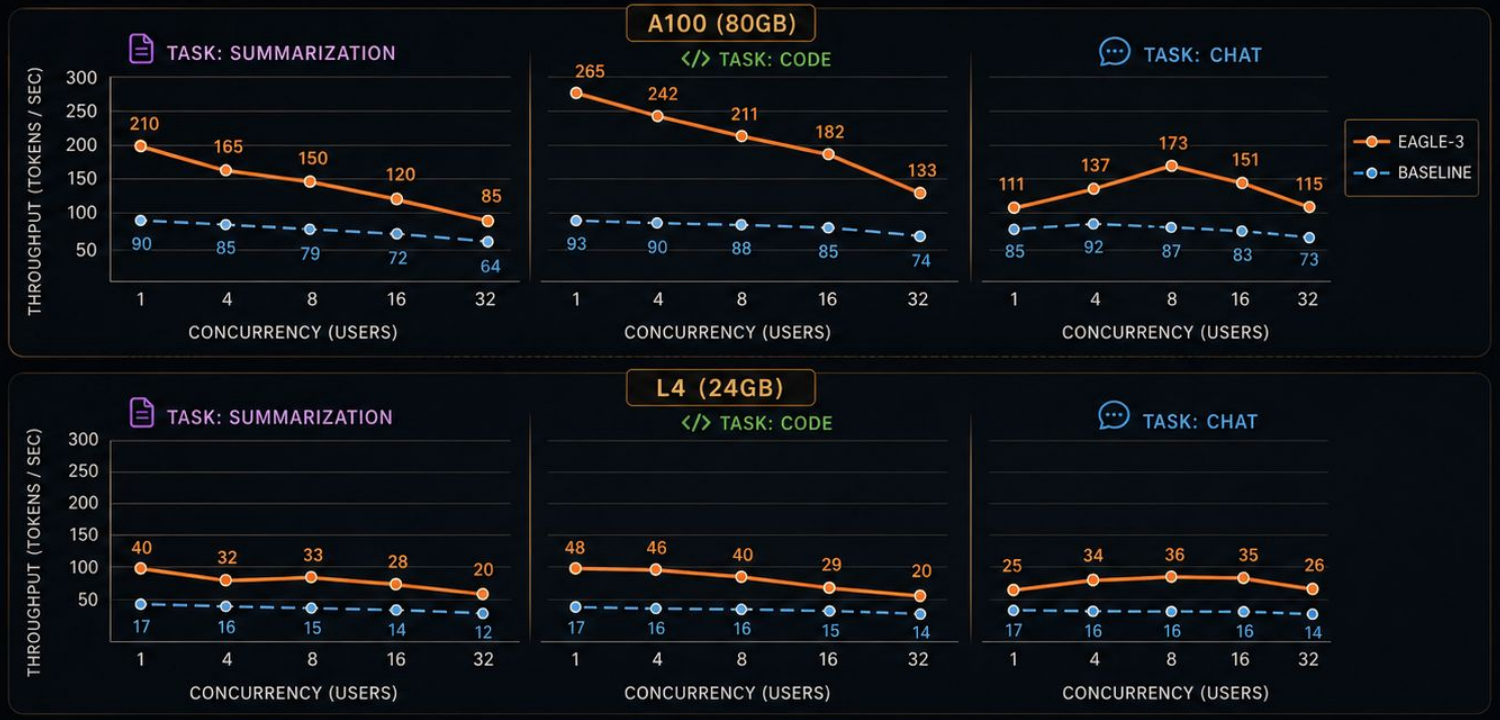
A100 (80GB)					
TASK	c=1	c=4	c=8	c=16	c=32
Chat (ShareGPT)	1.31×	1.50×	1.98×	1.79×	1.59×
★ Code (HumanEval)	2.87×	2.67×	2.38×	2.13×	1.80×
Summarization (CNN)	2.33×	1.88×	1.77×	1.55×	1.33×

L4 (24GB)					
TASK	c=1	c=4	c=8	c=16	c=32
Chat (ShareGPT)	1.20×	1.36×	1.61×	1.59×	1.43×
★ Code (HumanEval)	2.91×	2.73×	2.44×	2.12×	1.74×
Summarization (CNN)	2.27×	1.96×	1.81×	1.61×	1.28×

★ Peak: Code @ c=1 on L4 = 2.91× | Minimum: Chat @ c=1 on L4 = 1.20× | No crossover observed at any concurrency level — EAGLE-3 always wins

EAGLE-3 IS AN UNCONDITIONAL WIN (A100 & L4)

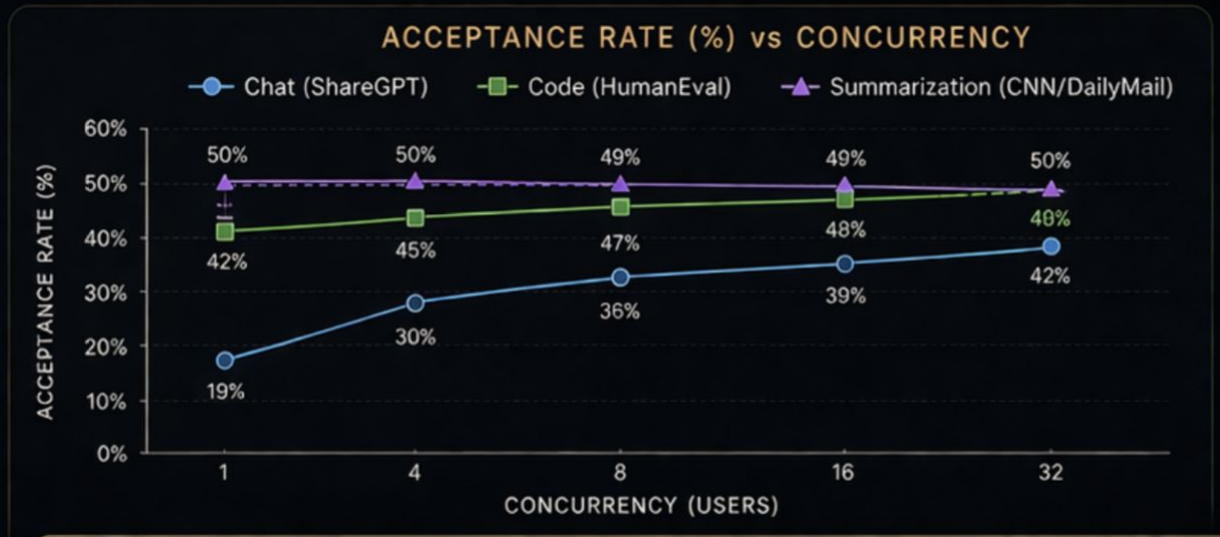
EAGLE-3 DELIVERS 2X–3X HIGHER THROUGHPUT ACROSS ALL CLOUD GPUS



NO CROSSOVER POINT OBSERVED — EAGLE-3 IS ALWAYS FASTER THAN BASELINE

Even at 32 concurrent requests, EAGLE-3 remains strictly dominant across all tasks and hardware.

WHY DOES ACCEPTANCE RATE CHANGE WITH CONCURRENCY?



CHAT (ShareGPT)
Rises dramatically
19% → 42%

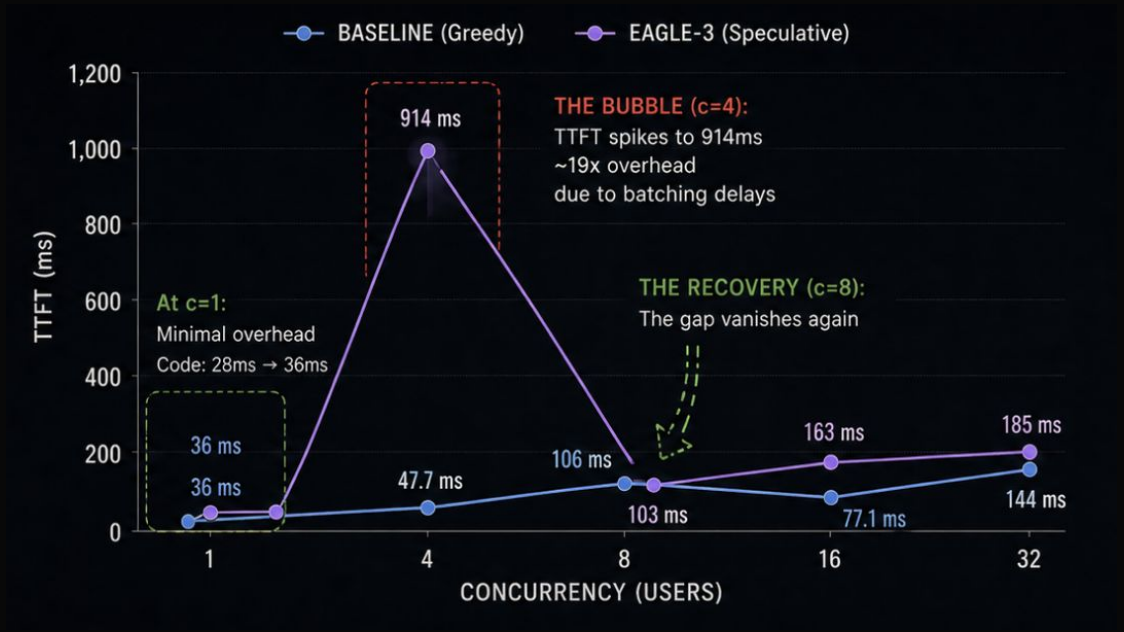
CODE (HumanEval)
Slowly rises (highly structured)
42% → 50%

SUMMARIZATION (CNN)
Gently falls
50% → 48%

ACCEPTANCE RATES ARE IDENTICAL ON A100 AND L4 – A PROPERTY OF THE MODEL PAIR, NOT THE HARDWARE.

WHEN DOES THE OVERHEAD MATTER?

TTFT (Time-to-First-Token) on A100 Across Concurrency Levels



1

AT c=1

TTFT overhead is minimal.
Code: 28ms → 36ms only.

2

THE BUBBLE

At c=4 (Chat), TTFT spikes to 914ms — ~19× overhead due to batching delays.

3

THE RECOVERY

By c=8, the gap vanishes again.
103ms vs 106ms — essentially equal.

TTFT Time to First Token (user-perceived latency)

TPOT Time Per Output Token (generation speed)

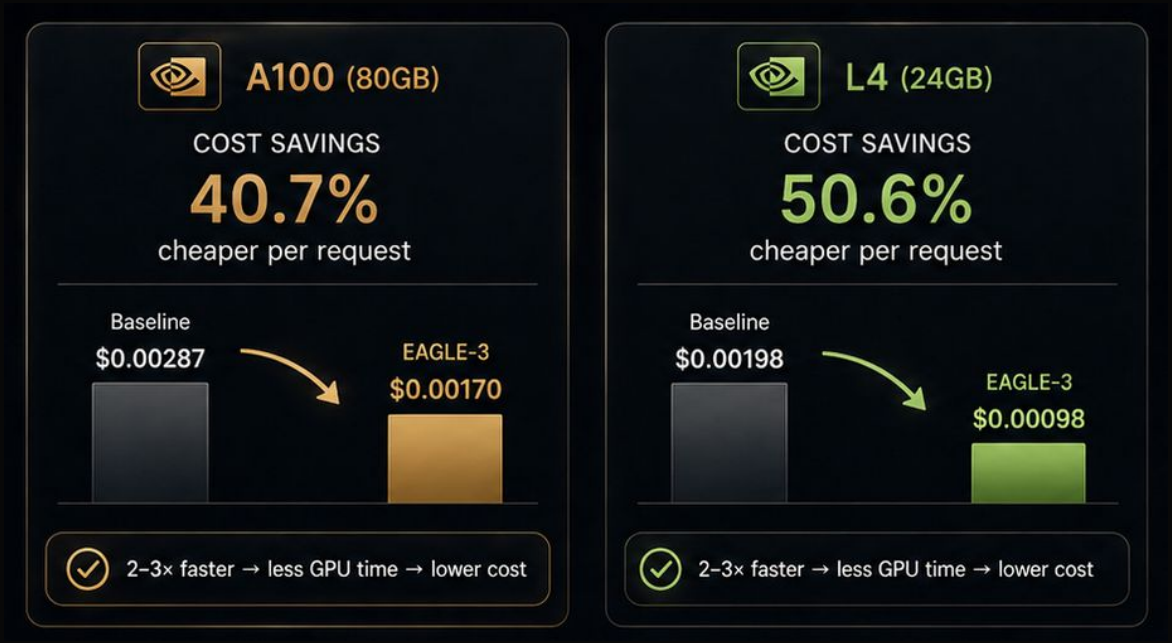
= Total Experience: Overhead vs sustained faster generation

TAKEAWAY

TPOT (generation speed) always improves, but initial TTFT has a scheduling bubble at low-medium concurrency.

CLOUD ECONOMICS: 50% CHEAPER COMPUTE

EAGLE-3 delivers 2–3× higher throughput ⟶ 40–50% lower cost per request



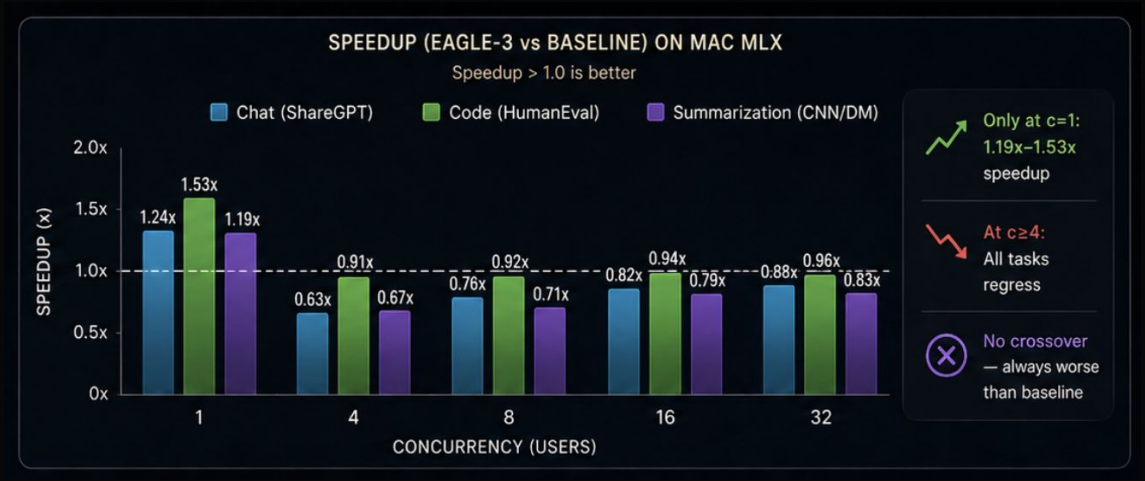
★ THE SWEET SPOT

L4 + EAGLE-3 matches A100 Baseline throughput (~90 vs ~87 TPS) at 66% lower cost ($0.00287 ⟶ $0.00098).

2–3× faster ⟶ less GPU time ⟶ lower cost. Maximum performance at minimum cost.

THE EDGE CASE: MAC MLX

Speculative decoding on Apple Silicon is a single-user win, but a multi-user failure.



TAKEAWAY: Mac MLX is best for single-user workloads. Under concurrency, it becomes unpredictable and unusable.

1 SINGLE-USER SUCCESS

Genuine 1.19×–1.53× speedup at c=1.
Best for local, single-user agents.

2 MULTI-USER CATASTROPHE

Regressions across all tasks at c≥4.
Throughput drops below baseline.

3 THE UNFAIRNESS PROBLEM

At c=32, CV = 188% (Extremely Unfair).
Fastest 10%: 150ms vs Slowest 10%: 45,000ms (45 sec!)

WHY DID THE MAC FAIL? SCHEDULING VS. ALGORITHMS

The difference is in how speculative decoding work is scheduled across concurrent requests.

✓ vLLM (CLOUD GPUs): CONTINUOUS BATCHING

Batches speculative decoding across requests ⟶ Stable at scale

HOW IT WORKS

All requests are batched together for both draft and verify passes. GPU stays busy.

RESULT AT SCALE

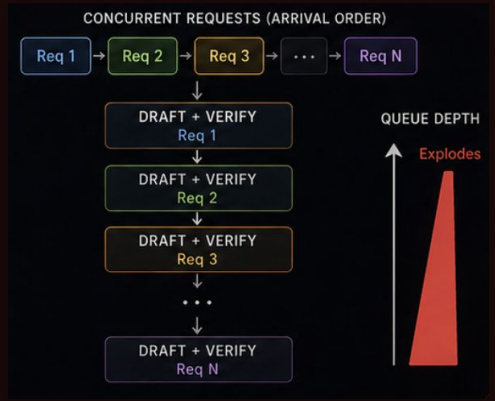
- ✓ High GPU utilization
- ✓ Stable TTFT & TPOT
- ✓ No queue explosion
- ✓ Scales to 32+ users

✕ MLX (MAC): SERIAL SPECULATIVE PASSES

Processes speculative passes serially ⟶ Queue depth explodes

HOW IT WORKS

Each request runs its entire draft+verify cycle before the next request begins. No cross-request batching.



RESULT AT SCALE

- ✕ Low GPU utilization
- ✕ TTFT becomes huge
- ✕ Massive queue buildup
- ✕ Regressions at c≥4

TAKEAWAY: The Mac regressions are an architectural scheduling failure, not a failure of speculative decoding theory.

FINAL TAKEAWAYS

Actionable conclusions from our rigorous experimentation

1 CLOUD GPUs (A100 / L4)

EAGLE-3 is an unconditional win.
Enable by default in production.

- 2×–3× higher throughput across all tasks
- Up to 2.91× speedup (Code, L4, c=1)
- 40–50% lower cost per request
- No crossover even at 32 concurrent users

TURN IT ON. LEAVE IT ON.
It's strictly better.

2 CODE GENERATION: BEST USE-CASE

Consistently delivers the highest gains.

- Peak 2.9× speedup on L4 at c=1
- Highest acceptance rates (42% → 50%)
- Most structured outputs = more predictable speculation
- First place for throughput, cost, and user experience

IF YOU WRITE CODE,
YOU WIN BIG WITH EAGLE-3.

3 EDGE DEPLOYMENTS (MAC MLX)

Strictly a single-user tool right now.

- Good at c=1 (1.19×–1.53× speedup)
- Collapses at c≥4 across all tasks
- Extreme unfairness at c=32 (CV 188%)
- Architectural scheduling bottleneck in MLX

USE FOR LOCAL, SINGLE-USER
AGENTS ONLY.

BOTTOM LINE: On vLLM + NVIDIA GPUs, EAGLE-3 is a **production-ready, cost-saving multiplier.**
On Apple Silicon, **wait for scheduler improvements** before using beyond single-user scenarios.